AI Tool for Generating Tamarin Rules

From Protocol Diagrams to Formal Verification Models

Task 1 Submission - Formal Verification of Security Protocols

Overview

Task 1 Objective

Given a protocol diagram (sequence of rounds), develop an **AI tool** that generates the Tamarin rules for the protocol.

Instructor: Prof. Raghavendra Singh Rohit, IIT Roorkee

This tool automatically converts **protocol sequence diagrams** (PNG, JPG, PDF) into **formal Tamarin verification models** (.spthy files) using a hybrid approach combining **Computer Vision (OCR)**, **Large Language Models (GPT-4)**, and **Template-Based Code Generation**.

☐ AI Pipeline Architecture

1

Input: Protocol Diagram

Accepts protocol diagrams in PNG, JPG, or PDF format showing message sequences between participants (Alice, Bob, Server, etc.)

↓

2

AI Component 1: Vision/OCR

Tesseract OCR + OpenCV extracts text, detects roles, identifies message arrows (→), and parses cryptographic operations (enc, dec, sign, verify)

↓

3

AI Component 2: LLM Enhancement

GPT-4 infers protocol semantics, detects fresh values (nonces, keys), and suggests security properties (authentication, secrecy lemmas)

↓

4

Template Generator

Applies Tamarin patterns from DigiLocker slides to generate syntactically correct rules with proper equational theories and action facts

↓

5

Output: Tamarin Model

Generates complete .spthy file ready for formal verification with Tamarin prover, including all rules and security lemmas

Key Features

Vision AI / OCR

Extracts protocol structure from diagram images using Tesseract OCR and computer vision

LLM Enhancement

GPT-4 infers semantics, fresh values, and security properties from extracted text

Template-Based

Reliable rule generation using patterns from DigiLocker formal verification slides

Security Lemmas

Automatically generates authentication and secrecy properties for verification

Multi-Format

Supports PNG, JPG, PDF diagrams and JSON protocol specifications

Tested & Verified

5/5 passing tests with example protocols and validation checks

How to Use

```
# Step 1: Install dependencies pip install pillow opencv-python pytesseract pymupdf openai # Step 2: Set API key (optional for AI enhancement) export OPENAI_API_KEY=sk-... # Step 3: Run the AI tool python task1_ai_tool.py protocol_diagram.png -o output.spthy # Step 4: Verify with Tamarin tamarin-prover output.spthy
```

Example: DigiLocker Aadhar Authentication

Input (Protocol Flow)

```
User → DigiLocker: Aadhaar + OTP DigiLocker → UIDAI: sign(aenc(AuthXML, pk_UIDAI), ltk_DigiLocker) UIDAI → DigiLocker: sign(aenc(Response, pk_DigiLocker), ltk_UIDAI) DigiLocker → User: Authenticated
```

Output (Generated Tamarin Rules)

```
theory DigiLocker_Aadhar_Authentication begin builtins: hashing, asymmetric-encryption, signing // PKI Setup rule RegisterKeys: [ Fr(~ltkD), Fr(~ltkA) ] --> [ !Ltk($DigiLocker, ~ltkD), !Ltk($UIDAI, ~ltkA), !Pk($DigiLocker, pk(~ltkD)), !Pk($UIDAI, pk(~ltkA)) ] // Protocol Execution rule DigiLocker_Send_Step2: let msg = sign(aenc(<~N, <~O, ~Tx>>, pkA), ltkD) in [ Fr(~N), Fr(~O), Fr(~Tx), !Ltk($DigiLocker, ltkD), !Pk($UIDAI, pkA) ] --[ SentAuth($DigiLocker, $UIDAI, ~N, ~O, ~Tx) ]-> [ Out(msg) ] // Security Properties lemma authentication: "All D A tx #i. TrustEstablished(D, tx) @ i ==> Ex #j. UIDAIResponded(A, D, tx) @ j & j < i" end
```

Task 1 Requirements Mapping

Requirement	Implementation	Status
Input: Protocol diagram (PNG, PDF)	`diagram_parser.py` with OCR	✓ Complete
Extract: Message sequences	Vision AI extracts roles, arrows	✓ Complete
Generate: Tamarin rules	`generator.py` with templates	✓ Complete
Handle: Key setup rules	`RegisterKeys` auto-generated	✓ Complete
Generic: Multiple protocols	Pattern-based extraction	✓ Complete

AI-based: "Develop an AI tool"	OCR + LLM enhancement	✓ Complete

View Source Code on GitHub

Complete source code, documentation, examples, and tests available

https://github.com/shamanthwick/tamarin-rule-generator

AI Technologies Used

Component	Technology	Purpose
Vision AI / OCR	Tesseract OCR + OpenCV	Extract text from diagrams, detect message flows
LLM Enhancement	OpenAI GPT-4	Infer semantics, detect fresh values, suggest lemmas
Pattern Recognition	Regex + Computer Vision	Identify crypto operations, roles, arrows
Code Generation	Template-Based (Python)	Generate syntactically correct Tamarin rules

Project Structure

```
tamarin-rule-generator/ ├── task1_ai_tool.py ← MAIN AI TOOL (Task 1 entry point) ├── src/ │ ├── diagram_parser.py ← Vision AI / OCR for diagram parsing │ ├── ai_analyzer.py ← LLM (GPT-4) semantic enhancement │ ├── parser.py ← JSON protocol parser │ ├── templates.py ← Tamarin rule templates │ └── generator.py ← Code generation engine ├── examples/ │ └── digilocker_aadhar.json ← Example protocol ├── tests/ │ └── test_generator.py ← Test suite (5/5 passing) ├── README.md ← Documentation ├── AI_USAGE.md ← AI component details ├── REPORT.md ← Technical report └── TASK1_SUMMARY.md ← Submission summary
```

Task 1 Submission - Formal Verification of Security Protocols

IIT Roorkee | Instructor: Prof. Raghavendra Rohit

GitHub Repository:

https://github.com/shamanthwick/tamarin-rule-generator

Generated: 2026 | MIT License